

WEEK-3 – Business Analytics

Task Name:

Multi-Echelon Supply Chain Network Design using Mixed-Integer Linear Programming (MILP)

1. Introduction

A multi-echelon supply chain moves products through more than one stage before they reach customers. In this project, factories send products to distribution centers, and the distribution centers deliver them to customer areas.

The aim is to choose a practical network that gives good service at a lower total cost. The model uses MILP so that both shipment quantities and distribution-center opening decisions can be handled together.

The simple flow is:

Factories → Distribution Centers → Customers

2. Objective

The main objective is to reduce the total cost of moving and handling products while meeting customer demand.

The model considers:

- Transportation cost from factories to distribution centers
- Transportation cost from distribution centers to customers
- Fixed cost of opening distribution centers
- Available factory capacity
- Distribution-center capacity
- Customer demand

This gives management a clear way to compare different network choices.

3. Supply Chain Network

A new example is used in this version. It contains three factories, three distribution centers, and four customer zones.

Factories

- Factory 1 (F1)
- Factory 2 (F2)
- Factory 3 (F3)

Distribution Centers

- Distribution Center 1 (D1)
- Distribution Center 2 (D2)
- Distribution Center 3 (D3)

Customer Zones

- Customer 1 (C1)
- Customer 2 (C2)
- Customer 3 (C3)
- Customer 4 (C4)

Products follow the path:

Factory → **Distribution Center** → **Customer**

4. Decision Variables

The model uses three simple decision variables.

x_{ij} – quantity sent from factory i to distribution center j .

y_{jk} – quantity sent from distribution center j to customer k .

z_j – binary variable showing whether distribution center j is open.

- $z_j = 1$ means the center is open.
- $z_j = 0$ means the center is closed.

The binary variable is important because the company must make a yes/no decision about opening each center.

5. Input Data

Factory capacities:

Factory	Maximum Capacity
F1	500 units
F2	700 units
F3	600 units

Distribution-center capacities and fixed costs:

Center	Capacity	Fixed Cost
D1	750 units	■4,200
D2	900 units	■5,000
D3	700 units	■3,800

Customer demand:

Customer	Demand
C1	400 units
C2	350 units
C3	550 units
C4	300 units

Total customer demand = 1,600 units.

Total factory capacity = 1,800 units.

Therefore, the available factory capacity is enough to cover the total demand.

6. Transportation Costs

Factory to distribution center cost per unit:

	D1	D2	D3
F1	■8	■13	■11
F2	■10	■7	■9
F3	■6	■11	■8

Distribution center to customer cost per unit:

	C1	C2	C3	C4
D1	■5	■8	■10	■9
D2	■8	■6	■7	■9
D3	■9	■10	■5	■6

These values are used by the optimization model to compare possible shipment routes.

A lower transportation cost is preferred, but capacity and demand constraints must also be satisfied.

7. Objective Function

The model minimizes three types of cost:

- Factory-to-distribution-center transportation cost
- Distribution-center-to-customer transportation cost
- Fixed cost of operating an open distribution center

The objective function is:

$$\text{Minimize } Z = \sum_i \sum_j c_{ij} x_{ij} + \sum_j \sum_k c_{jk} y_{jk} + \sum_j f_j z_j$$

In simple words, the model searches for the cheapest way to move all required products while deciding which distribution centers should operate.

8. Constraints

Constraint 1 – Factory Capacity

A factory cannot send more units than its available capacity.

$$\sum_j x_{ij} \leq S_i$$

Constraint 2 – Flow Balance

The quantity entering a distribution center must equal the quantity leaving it.

$$\sum_i x_{ij} = \sum_k y_{jk}$$

8. Constraints (continued)

Constraint 3 – Distribution Center Capacity

A center can handle products only up to its capacity. If it is closed, it cannot receive or send products.

$$\sum_i y_{ij} \leq H_j z_j$$

Constraint 4 – Customer Demand

Each customer must receive the required quantity.

$$\sum_i y_{ij} = D_j$$

Constraint 5 – Opening Decision

Each distribution center has a binary opening decision.

$$z_j \in \{0,1\}$$

Constraint 6 – Non-Negativity

Shipment quantities cannot be negative.

$$x_{ij}, y_{ij} \geq 0$$

Together, these constraints make sure that the solution is realistic and follows the business rules.

9. Python Implementation

The following simple Python model uses PuLP to represent the optimization problem.

```
import pulp

factories = ["F1", "F2", "F3"]
centers = ["D1", "D2", "D3"]
customers = ["C1", "C2", "C3", "C4"]

capacity = {"F1": 500, "F2": 700, "F3": 600}
dc_capacity = {"D1": 750, "D2": 900, "D3": 700}

fixed_cost = {"D1": 4200, "D2": 5000, "D3": 3800}

demand = {
    "C1": 400, "C2": 350,
    "C3": 550, "C4": 300
}

cost_fd = {
    ("F1", "D1"): 8, ("F1", "D2"): 13, ("F1", "D3"): 11,
    ("F2", "D1"): 10, ("F2", "D2"): 7, ("F2", "D3"): 9,
    ("F3", "D1"): 6, ("F3", "D2"): 11, ("F3", "D3"): 8
}

cost_dc = {
    ("D1", "C1"): 5, ("D1", "C2"): 8,
    ("D1", "C3"): 10, ("D1", "C4"): 9,
    ("D2", "C1"): 8, ("D2", "C2"): 6,
    ("D2", "C3"): 7, ("D2", "C4"): 9,
    ("D3", "C1"): 9, ("D3", "C2"): 10,
    ("D3", "C3"): 5, ("D3", "C4"): 6
}
```

9. Python Implementation (continued)

```
model = pulp.LpProblem(
    "Supply_Chain_Network",
    pulp.LpMinimize
)

x = pulp.LpVariable.dicts(
    "Factory_DC",
    [(i,j) for i in factories for j in centers],
    lowBound=0
)

y = pulp.LpVariable.dicts(
    "DC_Customer",
    [(j,k) for j in centers for k in customers],
    lowBound=0
)

z = pulp.LpVariable.dicts(
    "DC_Open", centers, cat="Binary"
)

model += (
    pulp.lpSum(cost_fd[i,j] * x[i,j]
                for i in factories for j in centers)
    +
    pulp.lpSum(cost_dc[j,k] * y[j,k]
                for j in centers for k in customers)
    +
    pulp.lpSum(fixed_cost[j] * z[j] for j in centers)
)

for i in factories:
    model += pulp.lpSum(x[i,j] for j in centers) <= capacity[i]

for j in centers:
    model += pulp.lpSum(x[i,j] for i in factories) == pulp.lpSum(y[j,k] for k in customers)
    model += pulp.lpSum(y[j,k] for k in customers) <= dc_capacity[j] * z[j]

for k in customers:
    model += pulp.lpSum(y[j,k] for j in centers) == demand[k]

model.solve(pulp.PULP_CBC_CMD(msg=False))
```

10. Expected Result

When the program is executed, the solver will return an optimized network. The output can include:

- Optimization status
- Minimum total cost
- Shipment quantities from factories to centers
- Shipment quantities from centers to customers
- Open or closed distribution centers

Example output format:

```
Status: Optimal
Minimum Total Cost: XXXXX

Factory to Distribution Center:
F1 -> D1 : ...
F2 -> D2 : ...
F3 -> D3 : ...

Distribution Center to Customer:
D1 -> C1 : ...
D2 -> C2 : ...
D3 -> C3 : ...

Distribution Center Decisions:
D1 Open: 1
D2 Open: 1
D3 Open: 0
```

The exact numbers are produced automatically by the solver based on the input data.

11. Business Interpretation

The optimized model can help a business answer practical questions such as:

- Which distribution centers should be opened?
- Which factory should supply each center?
- How many units should be transported on each route?
- Can all customer demand be served?
- Are any facilities close to their capacity limits?
- Which routes have lower transportation cost?

This approach reduces guesswork and supports faster planning. Managers can change capacities, costs, or demand values and run the model again.

12. Conclusion

This project shows how Mixed-Integer Linear Programming can be used for a multi-echelon supply chain network. The model combines transportation costs, fixed facility costs, capacity limits, and customer demand in one optimization problem.

By selecting suitable routes and distribution centers, a company can reduce unnecessary logistics costs while maintaining reliable product delivery.

Overall, MILP provides a simple data-driven method for making supply chain network decisions.